POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY

COURSE: INT1313 DATABASES - SEMESTER 1, 2026-2027

PHASE 1 PROGRESS REPORT - GROUP OF 3 STUDENTS

TOPIC 7: FOOD DELIVERY AGGREGATOR (MINI-DOORDASH)

- **Team Name:** G4
- **Team Members:** Trần Thanh Nhuận (n24dece085@student.ptithcm.edu.vn)
- Trần Hoàng Nguyên (n24dece084@student.ptithcm.edu.vn)
- Thái Tuấn Vũ (n24dece104@student.ptithcm.edu.vn)

1. INTRODUCTION & PROBLEM STATEMENT

The objective of this project is to architect a robust relational database for a Food Delivery Aggregator ecosystem. The system is designed to seamlessly connect restaurants, customers, and delivery drivers. The database must maintain data integrity while tracking the entire lifecycle of an order from placement and food preparation to delivery status.

Database Challenge: The core complexity lies in managing multi-way relationships across Customer, Restaurant, Driver, and Order tables. The design also supports location information for drivers and operational hubs.

2. SYSTEM REQUIREMENTS

2.1. User Requirements vs. System Capabilities

- **User Requirements:** Customers need to browse menus and place orders securely. Restaurants require an interface to manage menu items and accept orders. Drivers need a system to receive dispatch requests and update their location.
- **System Capabilities:** The database uses Extended ER (EER) concepts, including specialization/generalization, to manage a central User superclass and Customer, Restaurant, and Driver subclasses.

2.2. Functional Requirements (Data-Centric)

- The system must allow a Driver to update the status of an order, for example from PICKED_UP to DELIVERING.
- The system must support calculating order totals from OrderItem quantities and recorded item prices.
- Foreign-key relationships must preserve referential integrity between users, customers, restaurants, drivers, orders, menus, payments, and related entities.

2.3. Non-Functional Requirements

- **Performance:** Multi-way joins involving Orders, Users, and OrderItems should execute efficiently for dashboard and reporting workloads.
- **Security:** Sensitive customer data, particularly passwords and payment-related information, should be protected appropriately.

- **Integrity:** Relational mappings should follow normalization principles and minimize unnecessary redundancy.

3. BUSINESS RULES

The following business rules establish the fundamental constraints and operational logic governing the system.

3.1.Identity & Account Rules

- **BR-01 (Account Uniqueness):** A single phone number or email address can only be associated with one User account (uq_user_phone, uq_user_email).
- **BR-02 (Role Specialization Integrity):** Every Customer, Restaurant, and Driver entity must reference a valid User account via a unique foreign key (user_id). A single User cannot operate multiple sub-roles simultaneously.

3.2 Ordering & Cart Rules

- **BR-03 (Order Financial Snapshot Integrity):** When an order is finalized, item pricing (price) and names (item_name) must be explicitly frozen into OrderItem, and the delivery address details captured into delivery_address_snapshot on Order. Future modifications to menu items or customer default addresses must never alter historical order records.
- BR-04 (**(Minimum Order Amount Constraint):** An order can only be placed if the subtotal of its menu items meets or exceeds the restaurant's required minimum threshold (restaurant. min_order_amount).

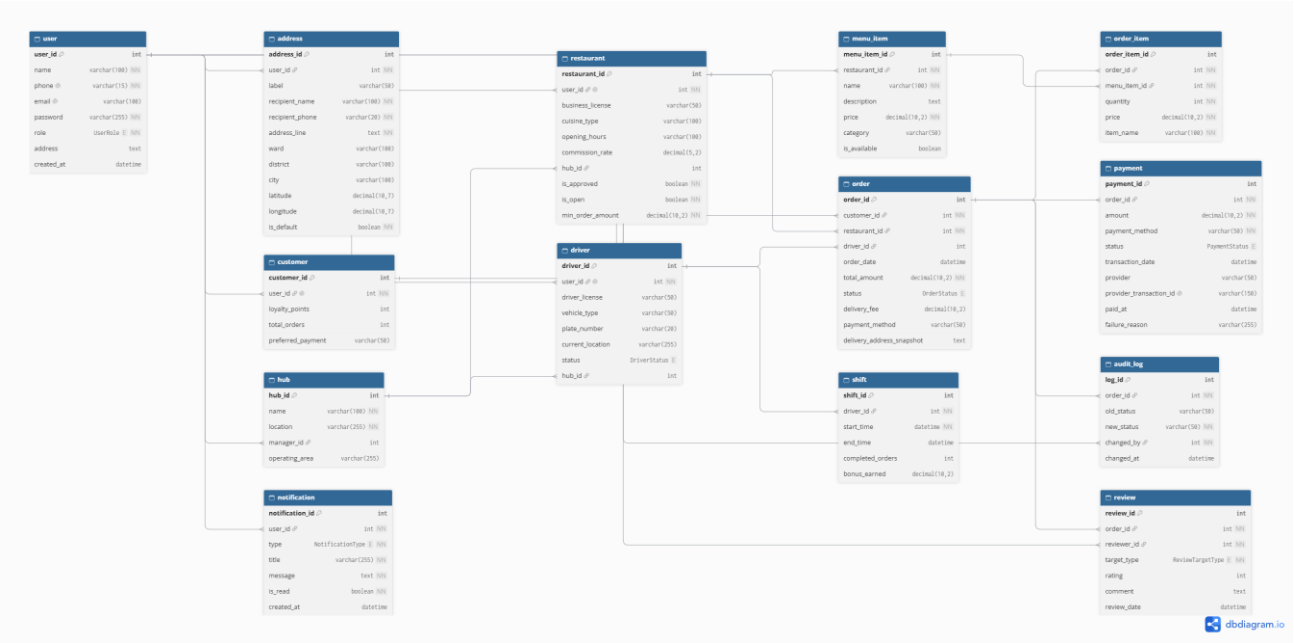
3.3 Fulfillment & Driver Logistics Rules

- **BR-05 (Order Assignment & Dispatch):** An order can only be assigned to an active driver (driver.status = 'ONLINE'). Upon assignment, the driver's operational status automatically updates to BUSY.
- **BR-06 (Hub Regional Assignment):** Restaurants and drivers are assigned to specific operational hubs (Hub). Drivers assigned to a hub are prioritized for order dispatch within that hub's coverage area (operating_area).
- **BR-07 (Shift & Bonus Reconciliation):** Driver shift performance, including start/end times (start_time, end_time), completed orders (completed_orders), and bonuses earned (bonus_earned), must be recorded per shift to ensure transparent income reconciliation and prevent dispatch discrepancies.

3.4 Governance & Audit Rules

- **BR-08 (Audit Log Immutability & Traceability):** Every transition in order status (PENDING => CONFIRMED => PREPARING => READY => PICKED_UP => DELIVERING => COMPLETED / CANCELLED) must create an immutable entry in AuditLog recording old_status, new_status, changed_by, and changed_at.
- **BR-9 (Review Eligibility & Target Constraint):** A review (Review) can only be submitted by a customer for an order that has reached the COMPLETED status. A single order can receive at most one review per target type (RESTAURANT or DRIVER).

4. ENTITY-RELATIONSHIP (ER) DIAGRAM & DATA DICTIONARY



4.1. User Inheritance Structure (IS-A Hierarchy)

The system implements a Superclass/Subclass architecture following EER principles. User acts as the central superclass, while Customer, Restaurant, and Driver represent specialized user roles. Admin does not have a separate table and is represented by User.Role = 'Admin'.

4.1.1. User – Superclass

Definition: Central entity for identity, contact, authentication, and roles.

Main attributes: UserID, Name, Phone, Email, Password, Role, Address, CreatedAt.

Relationship: Referenced by Address, Hub.ManagerID, AuditLog, CouponUsage, Notification, and Review.

Constraint / note: Centralizes shared user information.

4.1.2. Customer – Subclass

Definition: Represents customers who place food orders.

Main attributes: CustomerID, UserID, LoyaltyPoints, TotalOrders, PreferredPayment.

Relationship: User 1:0..1 Customer; Customer 1:N Order; Customer 0..1 Cart.

Constraint / note: CustomerID and UserID are identifiers; UserID is UNIQUE.

4.1.3. Restaurant – Subclass

Definition: Represents restaurant partners providing food.

Main attributes: RestaurantID, UserID, BusinessLicense, CuisineType, OpeningHours, CommissionRate, HubID, IsApproved, IsOpen, MinOrderAmount.

Relationship: User 1:0..1 Restaurant; Restaurant 1:N MenuItem and Order; Restaurant N:1 Hub.

Constraint / note: UserID is UNIQUE; HubID is nullable.

4.1.4. Driver – Subclass

Definition: Represents delivery partners.

Main attributes: DriverID, UserID, DriverLicense, VehicleType, PlateNumber, CurrentLocation, Status, HubID.

Relationship: User 1:0..1 Driver; Driver 1:N Order and Shift; Driver N:1 Hub.

Constraint / note: UserID is UNIQUE; DriverID in Order is nullable.

4.1.5. Admin

Definition: Administrative role represented in the User table.

Main attributes: User.Role.

Relationship: Identified by User.Role = 'Admin'.

Constraint / note: No separate Admin table exists in the current SQL.

4.1.6. Address

Definition: Stores delivery/contact addresses belonging to users.

Main attributes: AddressID, UserID, Label, RecipientName, RecipientPhone, AddressLine, Ward, District, City, Latitude, Longitude, IsDefault.

Relationship: User 1:N Address.

Constraint / note: Supports multiple saved addresses and a default address.

4.1.7. Hub

Definition: Represents an operational center/area.

Main attributes: HubID, Name, Location, ManagerID, OperatingArea.

Relationship: Hub 1:N Restaurant and Hub 1:N Driver; ManagerID references User.

Constraint / note: ManagerID is nullable.

4.1.8. Order

Definition: Represents an order created by a customer.

Main attributes: OrderID, CustomerID, RestaurantID, DriverID, OrderDate, TotalAmount, Status, DeliveryFee, Discount, PaymentMethod, DeliveryAddressSnapshot.

Relationship: Customer 1:N Order; Restaurant 1:N Order; Driver 0..1:N Order assignment.

Constraint / note: DriverID may be NULL.

4.1.9. AuditLog

Definition: Stores order-status change history.

Main attributes: LogID, OrderID, OldStatus, NewStatus, ChangedBy, ChangedAt.

Relationship: Order 1:N AuditLog; User 1:N AuditLog.

Constraint / note: Provides traceability for status changes.

4.1.10. MenuItem

Definition: Represents a food item offered by a restaurant.

Main attributes: MenuItemID, RestaurantID, Name, Description, Price, Category, IsAvailable.

Relationship: Restaurant 1:N MenuItem.

Constraint / note: Stores menu item information and availability.

4.1.11. Notification

Definition: Represents a notification sent to a user.

Main attributes: NotificationID, UserID, Type, Title, Message, IsRead, CreatedAt.

Relationship: User 1:N Notification.

Constraint / note: Type is ORDER, PROMOTION, or SYSTEM.

4.1.12. OrderItem

Definition: Represents a specific menu item in an order.

Main attributes: OrderItemID, OrderID, MenuItemID, Quantity, Price, ItemName.

Relationship: Order 1:N OrderItem; MenuItem 1:N OrderItem.

Constraint / note: Stores item price and name at order time.

4.1.13. Payment

Definition: Represents a payment transaction associated with an order.

Main attributes: PaymentID, OrderID, Amount, PaymentMethod, Status, TransactionDate, Provider, ProviderTransactionID, PaidAt, FailureReason.

Relationship: Order 1:N Payment in the current SQL because OrderID is not UNIQUE.

Constraint / note: Status is PENDING, SUCCESS, or FAILED. ProviderTransactionID is UNIQUE.

4.1.14. Review

Definition: Represents a user's review of a restaurant or driver.

Main attributes: ReviewID, OrderID, ReviewerID, TargetType, Rating, Comment, ReviewDate.

Relationship: User 1:N Review; Order 1:N Review.

Constraint / note: TargetType is Restaurant or Driver; ReviewerID references User.UserID.

4.1.15. Shift

Definition: Represents a driver's work shift.

Main attributes: ShiftID, DriverID, StartTime, EndTime, CompletedOrders, BonusEarned.

Relationship: Driver 1:N Shift.

Constraint / note: Supports driver workforce and shift management.

4.1.16. Entity Summary

Entity Group	Entity	Role

Superclass	user	Shared user information
Subclass	customer	Customer placing orders
Subclass	restaurant	Restaurant providing food
Subclass	driver	Delivery driver
Role in User	Admin	System administration
User Support	address	Delivery addresses
Management	hub	Operational center/area
Transaction	order	Customer order
Tracking	audit_log	Order-status history
Restaurant Menu	menu_item	Food item
Notification	notification	System/user notifications
Transaction Detail	order_item	Order-line details
Payment	payment	Payment transaction
Feedback	review	Restaurant/driver review
Driver Management	shift	Driver work shift

Main inheritance structure: User => Customer, User => Restaurant, User => Driver. Admin is identified through User.Role = 'Admin'.

4.2. DATA DICTIONARY

The following dictionary is aligned with the current MySQL SQL schema. It documents the actual data types, keys, nullability, defaults, foreign keys, and uniqueness constraints. Where the earlier report described a rule that is not present in the SQL, the SQL definition takes precedence here.

Table: user

Attribute	Data Type	Key	Nullable?	Default	Business Rules / Constraints & Examples
user_id	INT	PK	No	AUTO_ INCREMENT	Unique system-wide user identifier. *Example:* 1, 2, 3...
name	VARCHAR(100)	-	No	-	User's full name. *Example:* 'Tran Thanh Nhuan'
phone	VARCHAR(15)	-	No	-	UNIQUE; user phone number. ***Example:* '0912345678'**
email	VARCHAR(100)	-	Yes	NULL	UNIQUE; optional email address. *Example:* 'nhuan.nh24@gmail.com'
password	VARCHAR(255)	-	No	-	Password value; application should store it securely hashed. *Example:* '$2b$12$e8a88p...'
role	ENUM('Customer', 'Restaurant','Driver', 'Admin')	-	No	-	User role. *Example:* 'CUSTOMER'
address	TEXT	-	Yes	NULL	General contact or primary home address of the user. *Example:* '42 Pho Duc Chinh, Binh Thanh, Ho Chi Minh city '
created_at	DATETIME	-	Yes	CURRENT_ TIMESTAMP	Account creation timestamp. *Example:* '2026-03-15 08:30:00'

Table: address

Attribute	Data Type	Key	Nullable?	Default	Business Rules / Constraints & Example
address_id	INT	PK	No	AUTO_ INCREMENT	Unique address identifier. *Example:* 1, 2, 3...
user_id	INT	FK	No	-	References User(UserID) ON DELETE CASCADE.
label	VARCHAR(50)	-	Yes	NULL	Address label. *Example:* 'Home', 'Office'
recipient_name	VARCHAR(100)	-	No	-	Recipient name. *Example:* 'Tran Thanh Nhuan'
recipient_phone	VARCHAR(20)	-	No	-	Recipient phone number. *Example:* '0912345678'
address-line	TEXT	-	No	-	Main street/address details. *Example:* '42 Pho Duc Chinh'
ward	VARCHAR(100)	-	Yes	NULL	Ward. *Example:* '1 Ward'
district	VARCHAR(100)	-	Yes	NULL	District. *Example:* 'Binh Thanh'
city	VARCHAR(100)	-	Yes	NULL	City. *Example:* 'Ho Chi Minh City'
latitude	DECIMAL(10,7)	-	Yes	NULL	Latitude coordinate. *Example:* 10.8456123

longitude	DECIMAL(10,7)	-	Yes	NULL	Longitude coordinate. *Example:* 106.7945891
is_default	TINYINT(1)	-	No	-	Flag indicating if this is the default delivery address (1 = Default, 0 = Secondary).

Table: customer

Attribute	Data Type	Key	Nullable?	Default	Business Rules / Constraints & Examples
customer_id	INT	PK	No	AUTO_ INCREMENT	Unique customer identifier.
user_id	INT	FK	No	-	UNIQUE; references User(UserID). No ON DELETE clause is defined in the current SQL.
loyalty_points	INT	-	Yes	NULL	Customer loyalty points. *Example:* 250 pionts.
total_orders	INT	-	Yes	NULL	Stored total-order count. *Example:* 12
preferred_payment	VARCHAR(50)	-	Yes	NULL	Preferred payment method. *Example:* 'MOMO', 'CASH'

Table: hub

Attribute	Data Type	Key	Nullable?	Default	Business Rules / Constraints
hub_id	INT	PK	No	AUTO_ INCREMENT	Unique hub identifier.
name	VARCHAR(100)	-	No	-	Hub name. *Example:* 'Binh Thanh Hub 1'
location	VARCHAR(255)	-	No	-	Hub location. *Example:* '42 Pho Duc Chinh, Binh Thanh, Ho Chi Minh city '
manager_id	INT	FK	Yes	NULL	References User(UserID).
operating_area	VARCHAR(255)	-	Yes	NULL	Area managed by the hub. *Example:* 'Thu Duc City (5km radius)'

Table: restaurant

Attribute	Data Type	Key	Nullable?	Default	Business Rules / Constraints
restaurant_id	INT	PK	No	AUTO_ INCREMENT	Unique restaurant identifier.
user_id	INT	FK	No	-	UNIQUE; references User(UserID). No ON DELETE clause is defined.
business_license	VARCHAR(50)	-	Yes	NULL	Business license identifier. *Example:* 'BL-0312345678'
cuisine_type	VARCHAR(100)	-	Yes	NULL	Primary cuisine classification. *Example:* 'Vietnamese Rice & Noodles
opening_hours	VARCHAR(100)	-	Yes	NULL	Restaurant operating hours. *Example:* '06:00 - 21:00'
commission_rate	DECIMAL(5,2)	-	Yes	NULL	Restaurant commission rate. Current SQL has no DEFAULT 15.00. (represents 15%)
hub_id	INT	FK	Yes	NULL	References Hub(HubID); no ON DELETE SET NULL clause is defined.
is_approved	TINYINT(1)	-	No	-	Administrative approval status flag (1 = Approved, 0 = Pending).

is_open	TINYINT(1)	-	No	-	Real-time store availability flag (1 = Open for orders, 0 = Closed)
min_order_amount	DECIMAL(10,2)	-	No	-	Minimum order amount. *Example:* 30000.00

Table: driver

Attribute	Data Type	Key	Nullable?	Default	Business Rules / Constraints
driver_id	INT	PK	No	AUTO_ INCREMENT	Unique driver identifier.
user_id	INT	FK	No	-	UNIQUE; references User(UserID). No ON DELETE clause is defined.
driver_license	VARCHAR(50)	-	Yes	NULL	Driver license identifier. *Example:* 'DL-790123456789'
vehicle_type	VARCHAR(50)	-	Yes	NULL	Vehicle type. *Example:* 'Motorbike - Honda Wave Alpha'
plate_number	VARCHAR(20)	-	Yes	NULL	Vehicle registration plate. *Example:* '59-X3 123.45'
current_location	VARCHAR(255)	-	Yes	NULL	Latest reported GPS coordinates or location string. *Example:* '10.8451, 106.7941'
status	ENUM('ONLINE', 'OFFLINE', 'BUSY')	-	Yes	NULL	Driver's real-time operational state. *Example:* 'ONLINE'
hub_id	INT	FK	Yes	NULL	References Hub(HubID); no ON DELETE SET NULL clause is defined.

Table: order

Attribute	Data Type	Key	Nullable?	Default	Business Rules / Constraints & Examples
order_id	INT	PK	No	AUTO_ INCREMENT	Unique order identifier.
customer_id	INT	FK	No	-	References Customer(CustomerID).
restaurant_id	INT	FK	No	-	References Restaurant(RestaurantID).
driver_id	INT	FK	Yes	NULL	References Driver(DriverID); may be unassigned.
order_date	DATETIME	-	Yes	CURRENT_ TIMESTAMP	Order creation timestamp. *Example:* '2026-10-14 11:45:00'
total_amount	DECIMAL(10,2)	-	No	-	Stored total order amount. *Example:* 85000.00
status	ENUM('PENDING','CONFIRMED' ,'PREPARING' ,'READY' ,'PICKED_UP' ,'DELIVERING', 'COMPLETED' ,'CANCELLED')	-	Yes	NULL	Order lifecycle status. *Example:* 'DELIVERING'
delivery_fee	DECIMAL(10,2)	-	Yes	NULL	Calculated delivery charge based on routing distance. *Example:* 15000.00

payment_method	VARCHAR(50)	-	Yes	NULL	Name of the chosen payment method. *Example:* 'CASH', 'MOMO'
delivery_addres_snapshot	TEXT	-	Yes	NULL	Address snapshot captured for the order. *Example:* '42 Pho Duc Chinh, Binh Thanh, Ho Chi Minh city '

Table: order_item

Attribute	Data Type	Key	Nullable?	Default	Business Rules / Constraints
order_item_id	INT	PK	No	AUTO_ INCREMENT	Unique order-item identifier.
order_id	INT	FK	No	-	References Order(OrderID).
menu_item_id	INT	FK	No	-	References MenuItem(MenuItemID).
quantity	INT	-	No	-	Quantity of the specific menu item ordered (>0). *Example:* 2
price	DECIMAL(10,2)	-	No	-	Price recorded for the item in this order. *Example:* 40000.00
item_name	VARCHAR(100)	-	No	-	Item name recorded at order time. *Example:* 'Broken Rice with Grilled Pork

Table: audit_log

Attribute	Data Type	Key	Nullable?	Default	Business Rules / Constraints & Examples
log_id	INT	PK	No	AUTO_ INCREMENT	Unique audit-log identifier.
order_id	INT	FK	No	-	References Order(OrderID).
old_status	VARCHAR(50)	-	Yes	NULL	Previous order status. *Example:* 'PREPARING'
new_status	VARCHAR(50)	-	No	-	New order status. *Example:* 'READY'
changed_by	INT	FK	No	-	References User(UserID).
changed_at	DATETIME	-	Yes	CURRENT_ TIMESTAMP	Status-change timestamp. *Example:* '2026-10-14 12:00:15'

Table: menu_item

Attribute	Data Type	Key	Nullable?	Default	Business Rules / Constraints & Examples
menuItem_id	INT	PK	No	AUTO_ INCREMENT	Unique menu-item identifier.
restaurant_id	INT	FK	No	-	References Restaurant(RestaurantID).
name	VARCHAR(100)	-	No	-	Display name of the menu item *Example:* 'Broken Rice with Grilled Pork
description	TEXT	-	Yes	NULL	Detailed description of ingredients or preparation. *Example:* 'Grilled pork chop served with

					broken rice, steamed egg loaf, and sweet fish sauce'
price	DECIMAL(10,2)	-	No	-	Classification category for menu grouping. *Example:* 45000.00
category	VARCHAR(50)	-	Yes	NULL	Menu category. *Example:* 'Main Dish', 'Beverage'
is_available	TINYINT(1)	-	Yes	NULL	Real-time availability flag (1 = Available, 0 = Sold Out).

Table: notification

Attribute	Data Type	Key	Nullable?	Default	Business Rules / Constraints & Examples
notification_id	INT	PK	No	AUTO_ INCREMENT	Unique notification identifier.
user_id	INT	FK	No	-	References User(UserID) ON DELETE CASCADE.
type	ENUM('ORDER', 'PROMOTION', 'SYSTEM')	-	No	-	Notification category *Example:* 'ORDER'.
title	VARCHAR(255)	-	No	-	Concise title header of the notification. *Example:* 'Order is on its way!'
message	TEXT	-	No	-	Main text body/payload of the notification. *Example:* 'Driver Nguyen Van A is delivering your order #1001.
is_read	TINYINT(1)	-	No	-	Read status flag (1 = Read, 0 = Unread)
created_at	DATETIME	-	Yes	CURRENT_ TIMESTAMP	Notification creation timestamp. *Example:* '2026-10-14 12:05:00

Table: Payment

Attribute	Data Type	Key	Nullable?	Default	Business Rules / Constraints
payment_id	INT	PK	No	AUTO_ INCREMENT	Unique payment identifier.
order_id	INT	FK	No	-	References Order(OrderID). Not UNIQUE in current SQL.
amount	DECIMAL(10,2)	-	No	-	Monetary amount processed for the transaction. *Example:* 85000.00
payment_method	VARCHAR(50)	-	No	-	Payment method. *Example:* 'MOMO', 'CREDIT_CARD', 'CASH'
status	ENUM('PENDING','SUCCESS', 'FAILED')	-	Yes	NULL	Payment status *Example:* 'SUCCESS'.
transaction_date	DATETIME	-	Yes	CURRENT_ TIMESTAMP	Transaction timestamp. *Example:* '2026-10-14 11:45:10'
provider	VARCHAR(50)	-	Yes	NULL	Third-party payment gateway provider service name. *Example:* 'MomoGateway', 'VNPay'
provider_ transaction_id	VARCHAR(150)	-	Yes	NULL	UNIQUE provider transaction identifier. *Example:*

					'MOMO_TXN_987654321'
paid_at	DATETIME	-	Yes	NULL	Exact timestamp when the payment was successfully completed and verified. *Example:* '2026-10-14 11:45:25'
failure_reason	VARCHAR(255)	-	Yes	NULL	Reason for payment failure. Recorded error message or reason code if the payment transaction failed. *Example:* 'Insufficient account balance'

Table: review

Attribute	Data Type	Key	Nullable?	Default	Business Rules / Constraints
review_id	INT	PK	No	AUTO_ INCREMENT	Unique review identifier.
order_id	INT	FK	No	-	References Order(OrderID).
reviewer_id	INT	FK	No	-	References User(UserID).
target_type	ENUM('Restaurant','Driver')	-	No	-	Review target type. *Example:* 'DRIVER'
rating	INT	-	Yes	NULL	Numerical rating score given on a scale from 1 to 5. *Example:* 5
comment	TEXT	-	Yes	NULL	Customer's detailed review text feedback. *Example:* 'Driver delivered very fast and food was still hot!
review_date	DATETIME	-	Yes	CURRENT_ TIMESTAMP	Review creation timestamp. *Example:* '2026-10-14 12:30:00'
UNIQUE(order_ id, reviewer_id, target_type)	-	UQ	-	-	Prevents duplicate reviews for the same order, reviewer, and target type.

Table: shift

Attribute	Data Type	Key	Nullable?	Default	Business Rules / Constraints & Examples
shift_id	INT	PK	No	AUTO_ INCREMENT	Unique shift identifier.
driver_id	INT	FK	No	-	References Driver(DriverID).
start_time	DATETIME	-	No	-	Shift start time. *Example:* '2026-10-14 07:00:00
end_time	DATETIME	-	Yes	NULL	Shift end time. *Example:* '2026-10-14 12:00:00'
completed_orders	INT	-	Yes	NULL	Number of completed orders in the shift. *Example:* 8
bonus_earned	DECIMAL(10,2)	-	Yes	NULL	Bonus earned during the shift. *Example:* 50000.00

Relationship Summary

• User => Customer / Restaurant / Driver: each subclass has a UNIQUE, NOT NULL UserID foreign key, so each subclass row corresponds to exactly one User; a User may have zero or one

row in each subclass table. The current SQL does not enforce that the three subclass tables are mutually exclusive.

- Customer => Order: 1:N. Restaurant => Order: 1:N. Driver => Order: 1:N, with DriverID nullable on Order.
- Customer => Cart: 0..1 per customer because Cart.CustomerID is UNIQUE.
- Restaurant => MenuItem: 1:N. Hub => Restaurant: 1:N. Hub => Driver: 1:N.
- Order => OrderItem: 1:N. MenuItem => OrderItem: 1:N.
- Order => Payment: potentially 1:N in the current SQL because Payment.OrderID is not UNIQUE. If the business requirement is exactly one payment per order, add UNIQUE(OrderID).
- Order => Review: 1:N. User => Review: 1:N.
- Order => AuditLog: 1:N. User => AuditLog: 1:N.
- User => Notification: 1:N. User => Address: 1:N. Driver => Shift: 1:N.

5.3.SCHEMA DESIGN

USER

user_id	name	phone	email	password	role	adress	Created_at

ADDRESS

address_id	user_id	label	Recipient_name	Recipient_phone	Address_line	ward	city

latitude	longitude	label	Is_default

- user_id (FK) => trỏ tới **user(user_id)**

CUSTOMER

customer_id	user_id	Loyalty_point	Recipient_name	Total_orders	Preferred_payment

- user_id (FK) => trỏ tới **user(user_id)**

RESTAURANT

restaurant_id	user_id	Business_license	Cuisine_type	Opening_hours	Commission_rate

Hub_id	is_approved	is_open	min_order_amount

- user_id (FK) => trỏ tới **user(user_id)**
- hub_id (FK) => trỏ tới **hub(hub_id)**

DRIVER

driver_id	user_id	driver_license _license	vehicle_type	plate_number	current_location

status	Hub_id

- user_id (FK) => trỏ tới **user(user_id)**

- hub_id (FK) => trỏ tới **hub(hub_id)**

HUB

hub_id	name	location	manager_id	operating_area

- manager_id (FK) => trỏ tới **user(user_id)**

ORDER

order_id	customer_id	restaurant_id	driver_id	total_amount	status	delivery_fee

payment_method	delivery_address_snapshot

- customer_id (FK) => trỏ tới **customer(customer_id)**
- restaurant_id (FK) => trỏ tới **restaurant(restaurant_id)**
- driver_id (FK) => trỏ tới **driver(driver_id)**

ORDER_ITEM

Order_item_id	order_id	Menu_item_id	quantity	price	item_name

- order_id (FK) => trỏ tới **order(order_id)**
- menu_item_id (FK) => trỏ tới **menu_item(menu_item_id)**

AUDIT_LOG

log_id	order_id	old_status	new_status	Changed_by	Changed_at

- order_id (FK) $\rightarrow$ trỏ tới **order(order_id)**
- changed_by (FK) $\rightarrow$ trỏ tới **user(user_id)**

MENU_ITEM

Menu_item_id	restaurant_id	name	description	price	category	is_available

- restaurant_id (FK) => trỏ tới **restaurant(restaurant_id)**

NOTIFICATION

notification_id	user_id	type	title	message	status	is_read	created_at

- user_id (FK) => trỏ tới **user(user_id)**

REVIEW

review_id	order_id	Reviewer_id	target_id	rating	comment	review_date

- order_id (FK) => trỏ tới **order(order_id)**
- reviewer_id (FK) => trỏ tới **user(user_id)**

SHIFT

shift_id	driver_id	start_time	end_time	completed_orders	bonus_earned

- driver_id (FK) => trỏ tới **driver(driver_id)**

1.Nhóm Tài khoản & Kế thừa (Quan hệ 1:1) Đây là nhóm sử dụng mô hình thiết kế Superclass/Subclass, trong đó bảng `user` chứa thông tin chung (tên, sđt, email, mật khẩu), còn các bảng con chứa thông tin đặc thù.

- **`user` và `customer` (Khách hàng):** Liên kết 1:1. Một tài khoản đóng vai trò khách hàng sẽ có một bản ghi trong `customer` (lưu điểm `loyalty_points`). Khóa ngoại: `user_id` nằm ở `customer`.

- **`user` và `restaurant` (Nhà hàng):** Liên kết 1:1. Một tài khoản đóng vai trò đối tác quán ăn sẽ có một bản ghi trong `restaurant` (lưu giờ mở cửa, loại ẩm thực). Khóa ngoại: `user_id` nằm ở `restaurant`.

- **`user` và `driver` (Tài xế):** Liên kết 1:1. Một tài khoản đóng vai trò đối tác giao hàng sẽ có một bản ghi trong `driver` (lưu biển số xe, bằng lái). Khóa ngoại: `user_id` nằm ở `driver`.

2. Nhóm Vận hành & Hệ sinh thái (Quan hệ 1:N) Nhóm này định hình cơ sở vật chất và dữ liệu tĩnh của nền tảng.

- **`user` và `address` (Sổ địa chỉ):** Liên kết 1:N. Một người dùng (đặc biệt là khách hàng) có thể lưu nhiều địa chỉ giao hàng khác nhau (nhà riêng, công ty). Khóa ngoại: `user_id` nằm ở `address`.

- **`hub` và `restaurant` / `driver` (Khu vực quản lý):** Liên kết 1:N. Hệ thống chia thành các khu vực (Ví dụ: Hub Thủ Đức, Hub Quận 1). Một Hub quản lý nhiều Nhà hàng và Tài xế hoạt động trong đó. Khóa ngoại: `hub_id` nằm ở `restaurant` và `driver`.

- **`restaurant` và `menu_item` (Thực đơn):** Liên kết 1:N. Một quán ăn có rất nhiều món ăn trong menu. Khóa ngoại: `restaurant_id` nằm ở `menu_item`.

- **`driver` và `shift` (Ca làm việc):** Liên kết 1:N. Một tài xế đăng ký nhiều ca làm việc khác nhau trong tuần. Khóa ngoại: `driver_id` nằm ở `shift`.

3. Nhóm Giao dịch Cốt lõi (Quan hệ trung gian bẻ gãy M:N) Đây là nhóm trung tâm lưu lượng dữ liệu (Data Traffic) của toàn bộ hệ thống.

- **`order` (Đơn hàng):** Đóng vai trò là bảng trung gian khổng lồ kết nối 3 thực thể: Khách hàng, Nhà hàng và Tài xế. Một đơn hàng bắt buộc phải có `customer_id` (người mua) và `restaurant_id` (nơi bán). `driver_id` (người giao) có thể được cập nhật sau khi có tài xế nhận đơn.

- **`order_item` (Chi tiết đơn hàng):** Đóng vai trò bẻ gãy quan hệ Nhiều-Nhiều giữa Đơn hàng và Món ăn. Một `order_item` chứa khóa ngoại trỏ về `order_id` và `menu_item_id`, kèm theo `quantity` (số lượng) và `price` (giá chốt tại thời điểm mua).

4. Nhóm Ăn theo Đơn hàng (Quan hệ 1:1 và 1:N) Nhóm này chứa các dữ liệu phát sinh sau khi đơn hàng được tạo ra.

- **`order` và `payment` (Thanh toán):** Liên kết 1:1 hoặc 1:N (tùy nghiệp vụ, nếu cho phép thanh toán lỗi rồi quẹt thẻ lại thì là 1:N). Lưu trữ mã giao dịch, phương thức (MOMO, Tiền mặt) và trạng thái thanh toán. Khóa ngoại: `order_id` nằm ở `payment`.

- **`order` và `audit_log` (Lịch sử trạng thái):** Liên kết 1:N. Một đơn hàng từ lúc đặt đến lúc giao sẽ sinh ra nhiều dòng log (PENDING -> PREPARING -> DELIVERING ->

COMPLETED). Khóa ngoại: `order_id` nằm ở `audit_log`.

- **`order` và `review` (Đánh giá):** Liên kết 1:1. Khách hàng chỉ được để lại 1 lượt đánh giá (số sao, nhận xét) cho 1 đơn hàng đã hoàn thành. Khóa ngoại: `order_id` nằm ở `review`.